

HW5 writeup

B11902083 徐燦桓

For all problem except for Cosmic Ray, just run `solve.py` under respective directory to get flag.

[Crypto] Cosmic Ray

- Flag: `FLAG{overcoming_the_time_limit_is_the_hardest_part_right?}`

Solution

For this problem, we are provided with public key (n, e) from an RSA system, and two ciphertext c_1, c_2 encrypted from $m_1 = d + \delta_1, m_2 = d + \delta_2$ where $\delta_1, \delta_2 \in [0, 2^{20})$.

We can see that $m_2 = f(m_1)$ where $f(x) = x - \delta_1 + \delta_2$, meaning that we can apply Franklin-Reiter related-message attack to recover m_1, m_2 . The issue is that $b = \delta_2 - \delta_1 \in (-2^{20}, 2^{20})$ is unknown to us, which means we need to first brute-force the correct b by assumption, and check if the returned m_1, m_2 actually matches the ciphertext we get. For the `franklinReiter` implementation, I adapted the code in [this repo](#).

```
def test_delta(d):
    for sign in [1, -1]:
        delta = sign * d
        try:
            m1 = franklinReiter(delta)
        except ValueError:
            continue
        m2 = (m1 + delta) % n
        # sanity check
        if pow(int(m1), int(e), int(n)) == c1 % n and pow(int(m2), int(e), int(n)) == c2 % n:
            return (m1, delta)
    return None
```

Here, I limited the range of search to $[-500000, -100000] \cup [100000, 500000]$ based on observation that in most attempt b seems to be in this range.

After recovering b, m_1, m_2 we still need to determine the value of δ_1 to actually recover d . This one is nothing but brute-force, except that to speed up I rearranged the equation a little to avoid computing the power $c_1^{m_1}$.

```
def test_r_guess(r):
    # d + r = m1, c1^(m1 - r) = m1, c1^m1 = m1 * c1^r
    lhs = (gmpy2.mpz(m1) * gmpy2.powmod(gmpy2.mpz(c1), gmpy2.mpz(r), gmpy2.mpz(n))) % n
    if lhs == rhs:
        return r
    return -1
```

Finally, based on "[Handbook of Applied Cryptography](#)" (i) Relation to factoring on page 287, it is possible to recover the factors of n with d revealed. `rsatool` on GitHub has an implementation available for this algorithm described in the book which I also adapted to my code. With p, q , we only need to compute x such that $x^{0x1337} \equiv 42 \pmod n$, which is trivial.

To meet the time limit, the last thing I did was to apply `multiprocessing` on both `test_delta` and `test_r_guess` and additionally, move these computationally-heavy logic into a separate script `exploit.py` then run it a machine with more cores (in my case, it was the NTU CSIE workstations) Client-side code is in `client.py` and is run locally.

In summary, the step would be:

- Run `exploit.py` then `client.py`
- Copy `n,e,c1,c2` received at `client.py` to `exploit.py`
- Wait for `exploit.py` to output `p,q`, then copy that back to `client.py`
- If completed in time (i.e. `client.py` didn't terminate due to the 69 second limit) we can have the flag.

I tried a few time for this to work as the exploit heavily depends on whether we can quickly finish the Franklin-Reiter part. With a considerable possibility delta would appear early in our range and the rest can be completed in seconds.

```
[*] x = FLAG{overcoming_the_time_limit_is_the_hardest_part_right?}
[*] Closed connection to 10.113.0.1 port 35003
```

[Crypto] Hidden Curve

- Flag: `FLAG{just_factor_it_and_combine_the_results_back_using_crt}`

Solution

In this problem, we are given a ECDSA system over a custom curve of unknown parameter. The only value revealed are: Modulus of base ring p , the public key $Y = G^x$ and the generator G . We are also given the encryption `enc` of flag and the signature of that encryption `sig`.

First, we always inspect whether the modulus p is indeed prime, as if it is a composite, the constructed curve would be a [pseudocurve](#) that can be reduced into multiple curves over rings of its factors. After solving ECDLP in these easier curves, we can reconstruct the solution in the original curve with CRT. Looking up p on FactorDB, we can see it is indeed a composite ($p = p_1 \times p_2$):

Result:		
status (?)	digits	number
FF	154 (show)	4841047389...91 <154> = 5032843102...43 <77> · 9618911796...37 <77>

Secondly, from the value of G, Y we can actually recover the coefficients of the secret curve:

$$\begin{aligned}G_y^2 &= G_x^3 + aG_x + b \pmod{p} \\ Y_y^2 &= Y_x^3 + aY_x + b \pmod{p} \\ a &= ((G_y^2 - Y_y^2) - (G_x^3 - Y_x^3))(G_x - Y_x)^{-1} \pmod{p} \\ b &= G_y^2 - G_x^3 - aG_x\end{aligned}$$

Then, it suffices to solve ECDLP in the sub-curves. If we're lucky that both curve satisfy certain property for ECDLP to be solvable, we can have the private key x in the original curve.

Curve 1: Singular (node)

With a simple check, we can see the curve over p_1 is a singular curve of node $y^2 = (x - \alpha)^2(x - \beta)$.

```
delta_1 = (4 * a**3 + 27 * b**2) % p1
# Delta mod p1: 0
```

which allows us to reduce this ECDLP to $(\mathbb{F}_{p_1}, \times)$ by the homomorphism $\phi: E \rightarrow \mathbb{F}_{p_1}$.

$$\phi(x, y) = \frac{y + \sqrt{\alpha - \beta}(x - \alpha)}{y - \sqrt{\alpha - \beta}(x - \alpha)}$$

The part for constructing homomorphism in my code references [this repo](#). After mapping the points to $(\mathbb{F}_{p_1}, \times)$, we can use CADO-NFS which is capable of solving 256-bit DLP in the multiplicative group on a home PC.

Note that since we cannot choose the generator CADO-NFS gives us, we will have to instead compute the DLP solution of both $\phi(G), \phi(Y)$ respective to this generator, then divide the solutions to get the solution x' for $\phi(G)^{x'} \equiv \phi(Y) \pmod q$. In summary, we should run:

```
./cado-nfs.py -dlp -ell <F1_G_ord> target=<F1_Y> <p_1> # a
./cado-nfs.py -dlp -ell <F1_G_ord> target=<F1_G> <p_1> # b
```

where `F1_G_ord` is the multiplicative order $q = \text{ord}(\phi(G))$, `F1_Y`, `F1_G` are $\phi(Y), \phi(G)$.

The following images shows the output of first and second command, where the result is the number after $\log(\text{target})$, and the generator chosen by CADO-NFS is `logbase`.

```
Info:root: logbase = 9249902511317520539369019028089249675895818166660712858896671006603941932947
Info:root: target = 38959670137312767933650813065427283399855838010266910083864778197042748956971
Info:root: log(target) = 1228144487186139510923662539096265408920356780452156249662427068835639125896 mod ell
1228144487186139510923662539096265408920356780452156249662427068835639125896
Info:root: logbase = 9249902511317520539369019028089249675895818166660712858896671006603941932947
Info:root: target = 17205398144776127600842507132320445962256558316494391832936826823587629399326
Info:root: log(target) = 12909583811405021869722521969874084876324586974534525257235344464346852184341 mod ell
12909583811405021869722521969874084876324586974534525257235344464346852184341
```

After getting a, b from two runs (which I have hardcoded inside my script) $x' \equiv a \times b^{-1} \pmod q$ (Not $\pmod{p_1}$ since we're doing DLP in the multiplicative group)

Curve 2: Anomalous

Checking the order of the second curve, we found out it is the same as the order of its underlying ring.

```
F2 = GF(p2)
E2 = EllipticCurve(F2, [a, b])
E2_order = E2.order()
# E2_order == p2
```

This means it is an anomalous curve, and according to the slide we use Smart's attack to solve ECDLP. According to [this document](#), the attack is performed by "lifting" the points to a new curve over the p-adic numbers i.e. numbers in the form of $\sum_i a_i p^i, 0 \leq a_i < p$.

Here, I directly adapted the code from [this stackexchange answer](#). What this code does is basically defining another `EllipticCurve(Qp(p, 2), [ZZ(t) + randint(0, p)*p for t in E.a_invariants()])`, which basically turn the curve's coefficient a, b into $a + kp, b + k'p$ with $0 \leq k, k' < p$. Next, use `lift_x` to lift G, Y to the new curve. Finally, we can compute the ECDLP solution x'' for $G^{x''} \equiv Y \pmod{p_2}$ by computing the quotient of the $G.x/G.y$ and $Y.x/Y.y$.

CRT & Decryption

With both x' , x'' computed, just combine the both with CRT. The only thing to notice is that x' is modulo $q = \text{ord}(\phi(G)) \neq p_1$. After getting the full private key x , the decryption steps are just following the Elgamal scheme:



Compute the shared key

$$K_s = K_E^d \pmod{p}$$

Decrypt the message

$$x = y \times K_s^{-1} \pmod{p}$$

40

```
# enc = [elgamal tpk (128)][nonce (8)][encrypt(flag) (?)]
E = EllipticCurve(R, [a, b])

tpk_bytes = enc[:128]
nonce = enc[128:136]
ct = enc[136:]

Tx = int.from_bytes(tpk_bytes[:64], "big")
Ty = int.from_bytes(tpk_bytes[64:], "big")
T = E(Tx, Ty) # tpk, or the ephemeral key

# Compute the shared key
S = sk * T
S_bytes = int(S[0]).to_bytes(64, "big") + int(S[1]).to_bytes(64, "big")

# Decrypt the message
# cipher = AES.new(sha256(key).digest(), AES.MODE_CTR)
key = sha256(S_bytes).digest()
cipher = AES.new(key, AES.MODE_CTR, nonce=nonce)
m = cipher.decrypt(ct)
# FLAG{just_factor_it_and_combine_the_results_back_using_crt}
```

```
Discrete log over F1: 13928748140329353458149027386124682916313564119545797633485277639327040560338
Curve order over F2: 96189117962437811316056177337022322304400484589113136985607890679543273441737
Discrete log over F2: 633611973020285063420650557666440961149492191132396929125460993238139409174
Private key mod l: 306629121536533750828732364488519194344566202583989509839070619229196974652194169769
FLAG{just_factor_it_and_combine_the_results_back_using_crt}
```

[Crypto] One Key

- Flag: FLAG{extending_wiener's_attack}

Solution

For this problem, we are given 42 public key pair (n_i, e_i) that is used to repeatedly encrypt `flag.txt` 42 times. The final result after 42 encryption is also given to us. We want to recover d first as it is used repeatedly over the rounds. Additionally, we can notice that d is way smaller in scale comparing to p, q, n for being a 481-bit prime. This hints that we can probably construct this as a lattice problem, and use LLL to find a short vector containing d .

Consider the equation of each encryption, we have:

$$\begin{aligned} e_i \cdot d &\equiv 1 \pmod{\varphi(n_i)} \\ \implies e_i \times d - k_i \times \phi(n_i) &= 1, \text{ for some } k_i \in \mathbb{Z}^+ \\ \implies e_i \times d - k_i \times n_i &= 1 - k_i \times (p_i + q_i - 1) \ll n_i \end{aligned}$$

Constructing the lattice based on the following basis:

$$B = \begin{pmatrix} W & e_1 & e_2 & \cdots & e_{42} \\ 0 & -n_1 & 0 & \cdots & 0 \\ 0 & 0 & -n_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -n_{42} \end{pmatrix}$$

and notice that the linear combination $[d, k_1, k_2, \dots, k_{42}]$ will generate the vector

$v = (dW, e_1d - k_1n_1, e_2d - k_2n_2, \dots, e_{42}d - k_{42}n_{42})$. The tailing terms are all of size $O(\sqrt{n_i})$, which is small considering vectors in this lattice have size $O(n_i)$ terms. The scaling factor W is to make the first term having similar scale as other terms, so as to make LLL find it. (But in fact the algorithm still works in this case even if $W = 1$) Running LLL on this basis should give us v as the shortest vector `LLL[0]` in the reduced basis `LLL[]`, where d can then be extracted by `d = LLL[0][0]/W`.

To recover the original message, we need to note that during the encryption, if for some $i > 1$ we had $n_{i-1} > n_i$, then some of the information will be lost when the encryption on i -th round computes modulo n_i . Therefore, after we reverse one encryption on i -th round, we also need to consider multiples of n_i as candidates and continue decryption from all these candidates. We check all candidates that reached the final layer and see which one contains `flag` in the decoded message.

```
FOUND FLAG: FLAG{extending_wiener's_attack}
Candidate: b'\xb4\xe8=\x84\x99r\xdf\x15\x04\x1d&E\x97\x1f\x8e\xe9\xe6\xde\xd9Q' ...
Candidate: b'\xcd\r\xdb\xcb#%c.: \xc4\x93\x1f\x1aP\x19\x1e\x98\xc9- ' ...
Candidate: b'\xa2\xbe1\x81\xbb&\x8c\x93K\x8f\xca\x8e4\x00\x99\xdd\x99\xfd\xd2\x03' ...
Candidate: b'\x96\xdc\]\0c\xbf\xab\x9dgf>\xf4\xbc:Y\xee6\xbc- ' ...
```

[Crypto] Not Random

- Flag: `FLAG{(ec)dsa_deterministic_nonces_should_use_RFC6979}`

Solution

For this problem, we are given a list of signatures `sigs` on 4 messages signed by a ECDSA key pair `(sk, pk)`, and a message encrypted by AES whose key is derived from `sk` deterministically. Thus, we should try to recover `sk` first.

The problem of this ECDSA scheme is that the nonce k is computed deterministically from the private key and the message:

```
k = (
    int.from_bytes(hmac.new(key, msg, hashfunc).digest()) % curve.q
)
```

Such "biased nonce" is one variant of the schemes that allow us to regard it as the Hidden Number Problem according to [this document](#). Again, we consider the signing equations (let `sk` be d):

$$\begin{aligned} s &\equiv k^{-1}(z + r \times d) \pmod{q} \\ \implies sk &= z + r \times d \pmod{q} \\ \implies k &= s^{-1}rd + s^{-1}z \pmod{q} \\ \implies k - t \times d + a &= 0 \pmod{q}, \text{ where } t = s^{-1}r, a = -s^{-1}z \end{aligned}$$

HNP states that:

Definition 4.10 (Hidden Number Problem). Let p be a prime and let $\alpha \in [1, p-1]$ be a secret integer. Recover α given m pairs of integers $\{(t_i, a_i)\}_{i=1}^m$ such that

$$\beta_i - t_i \alpha + a_i = 0 \pmod{p}$$

where the β_i are unknown and satisfy $|\beta_i| < B$ for some $B < p$.

We can match the above equation with the parameters here, as we indeed have:

- d being the secret integer, q being the prime
- $\{(t_i, a_i)\}_{i=1}^4$ can be computed from `sigs`
- β_i are the nonces and are output of `sha256`, satisfying the constraint $\exists B, |\beta_i| < B < q$

Then, it suffices to follow the formulation of basis and target vector based on the parameters we have for this HNP instance, as described in the same document.

With the SVP approach, we embed the CVP target vector as a row in the lattice basis to get the basis $\mathbf{B}'$:

$$\mathbf{B}' = \begin{bmatrix} p & & & & & \\ & p & & & & \\ & & \ddots & & & \\ & & & p & & \\ t_1 & t_2 & \cdots & t_m & B/p & \\ a_1 & a_2 & \cdots & a_m & & B \end{bmatrix}$$

This lattice contains the vector

$$\mathbf{u}' = (\beta_1, \dots, \beta_m, \alpha B/p, -B)$$

generated by the linear combination $(k_1, \dots, k_m, \alpha, -1)$. We have $\|\mathbf{u}'\| < \sqrt{m+2}B$ and so Proposition 3.15 suggests that we are likely to find this short vector among the basis vectors of an LLL-reduced basis. Notably, the shorter vector $(0, \dots, 0, B, 0)$ is in this lattice, generated by the linear combination $(-t_1, \dots, -t_m, p, 0)$, so $\mathbf{u}'$ is more likely to be the second vector of an LLL-reduced basis.

What I did is just plug in $p = q, t_i = s_i^{-1}r, a_i = -s_i^{-1}z$. To avoid the fraction term at B/p , I decide to scale the matrix by $S = q/B = 2^{128}$ such that we would instead have:

$$B = \begin{bmatrix} qS & 0 & \cdots & 0 & 0 & 0 \\ 0 & qS & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & qS & 0 & 0 \\ St_1 & St_2 & \cdots & St_n & 1 & 0 \\ Sa_1 & Sa_2 & \cdots & Sa_n & 0 & q \end{bmatrix}$$

Similarly, the target vector would become $[Sk_1, \dots, Sk_n, d, q]$ so we can still get d . Also, as described in the procedure, we should take `LLL[1]` in the reduced basis as the first one is very likely $(0, 0, \dots, q, 0)$, which is also confirmed in the printed output of `LLL[0]`, `LLL[1]` below. Finally, to get the flag, just follow the procedure to setup AES and decrypt `ct`.

```
Reducing lattice...
(0, 0, 0, 0, 39402006196394479212279040100143613805079739270465446667946905279627659399113263569398956308152294913554433653942643, 0) (2916985837
36204106469667976035847296471844462992958946908569600, 315468439737032401215604504211129222372393288337834594000970992074591134792643995953471108
4111639313074888437158056299948099737812580059892874781252098720515335848355189858209103872, 3203527179386762645768259415982450220640614285903257
14526148608, -17643873703713040892230979726619155930010283646636041472075072035641412083924243955231594199789262513766386891325164, 3940200619639
27659399113263569398956308152294913554433653942643)
sk = 21758132492681438320048060373524457875069455623829405195871833243986247315189019614167362108363032399788046762617479
Decrypted: b'FLAG{(ec)dsa_deterministic_nonces_should_use_RFC6979}'
```